Projekt semestralny – Informatyka IV rok

Instrukcja dla studentów i prowadzącej

Rok akademicki 2026/2027 • semestr 7 • praca zespołowa jak w firmie IT

Ten dokument opisuje organizację dużego projektu semestralnego realizowanego przez studentów w zespołach.

1. Cel zajęć

Celem zajęć jest przeprowadzenie pełnego procesu realizacji projektu informatycznego: od analizy potrzeb, przez wymagania, projekt, implementację, testy, dokumentację, aż do końcowego demo.

Studenci pracują jak mały zespół projektowy w firmie IT. Oceniany jest nie tylko końcowy program, ale również proces pracy, podział zadań, dokumentacja, testy, repozytorium Git i indywidualny wkład każdego członka zespołu.

Projekt trwa przez cały semestr. Cotygodniowe postępy są punktowane, dlatego nie wystarczy oddać projektu dopiero na końcu.

2. Podział na zespoły

Zespół	Student	Rola startowa	Uwagi

3. Tematy projektów

Projekt A: ServiceDesk – system obsługi zgłoszeń IT

System do rejestrowania i obsługi zgłoszeń użytkowników, podobny do firmowego helpdesku. Użytkownik zgłasza problem, technik obsługuje zgłoszenie, administrator zarządza słownikami i użytkownikami.

Elementy obowiązkowe:

- logowanie i role: User / Technician / Administrator
- CRUD zgłoszeń, priorytety, statusy, komentarze
- historia zmian i przypisanie osoby odpowiedzialnej
- dashboard statystyk, filtrowanie i wyszukiwanie
- baza danych, REST API, walidacja danych, testy

Możliwe rozszerzenia: powiadomienia e-mail, eksport raportów, dodatkowe dashboardy, załączniki, lista oczekujących, automatyczna klasyfikacja lub prosty moduł analityczny.

Projekt B: SmartBooking – system rezerwacji zasobów

System do rezerwacji sal, sprzętu, stanowisk albo innych zasobów organizacji. Najważniejsze jest wykrywanie konfliktów rezerwacji i czytelny kalendarz dostępności.

Elementy obowiązkowe:

- logowanie i role: User / Manager / Administrator
- lista zasobów, kalendarz dostępności, rezerwacje
- blokada konfliktów terminów i anulowanie rezerwacji
- historia rezerwacji, statusy, dashboard wykorzystania
- baza danych, REST API, walidacja danych, testy

Możliwe rozszerzenia: powiadomienia e-mail, eksport raportów, dodatkowe dashboardy, załączniki, lista oczekujących, automatyczna klasyfikacja lub prosty moduł analityczny.

Projekt C: CyberIncident – system zgłaszania incydentów bezpieczeństwa

System do zgłaszania i obsługi incydentów bezpieczeństwa, takich jak phishing, malware, utrata urządzenia lub podejrzenie wycieku danych.

Elementy obowiązkowe:

- logowanie i role: Employee / Security Analyst / Administrator
- zgłaszanie incydentów, kategorie, poziom zagrożenia
- przypisanie analityka, statusy, komentarze
- historia obsługi i dashboard incydentów
- baza danych, REST API, walidacja danych, testy

Możliwe rozszerzenia: powiadomienia e-mail, eksport raportów, dodatkowe dashboardy, załączniki, lista oczekujących, automatyczna klasyfikacja lub prosty moduł analityczny.

4. Zasady pracy jak w firmie IT

Repozytorium Git

- cały kod musi być w repozytorium
- każdy student powinien mieć własne commity
- nie przyjmujemy samego ZIP-a bez historii pracy

Backlog zadań

- każde zadanie ma nazwę, opis, właściciela, status i kryterium ukończenia
- statusy: To do, In progress, Review, Done
- zadania muszą być powiązane z wymaganiami REQ-xxx

Cotygodniowe review

- co zespół zrobił od poprzedniego spotkania
- co zespół zrobi podczas zajęć
- jakie są problemy i ryzyka
- krótkie demo działającego fragmentu systemu

Definition of Done

- funkcja działa
- kod jest w repozytorium
- zadanie jest przetestowane
- dokumentacja została zaktualizowana
- zespół umie pokazać i wyjaśnić funkcję

5. Co obowiązkowo musi być w dokumentacji

Dokumentacja jest obowiązkową częścią projektu. Nie może być dopisana dopiero po zakończeniu pracy. Powinna być aktualizowana po każdym sprincie.

Sekcja	Co ma zawierać	Czy jest obowiązkowe?
Informacje o projekcie	nazwa projektu, autorzy, wersja dokumentacji, historia zmian	TAK
Cel i zakres	problem, który rozwiązuje system; zakres MVP; elementy poza zakresem	TAK
Interesariusze i użytkownicy	opis ról użytkowników i ich potrzeb	TAK
Wymagania funkcjonalne	wymagania REQ-001, REQ-002 itd.; każde wymaganie musi być testowalne	TAK
Wymagania niefunkcjonalne	bezpieczeństwo, wydajność, niezawodność, użyteczność, kompatybilność	TAK
Przypadki użycia	scenariusze główne i alternatywne dla najważniejszych operacji	TAK
Architektura systemu	technologie, komponenty, diagram architektury, decyzje projektowe	TAK
Model danych	ERD, tabele/encje, relacje, najważniejsze pola	TAK
API / interfejsy	endpointy, parametry, odpowiedzi, kody błędów	TAK
Interfejs użytkownika	makiety lub zrzuty ekranów najważniejszych widoków	TAK
Testy	przypadki testowe, wyniki testów, lista błędów	TAK
Instrukcja instalacji	kroki uruchomienia projektu od zera	TAK
Instrukcja użytkownika	jak korzystać z systemu	TAK
Znane problemy	ograniczenia, błędy, funkcje niedokończone	TAK

6. Harmonogram i rozliczanie tygodniowe

Tydz.	Etap	Zakres pracy	Co oddać	Maks. pkt
1	Kick-off projektu	zespoły, role, repozytorium, wybór projektu	Repozytorium + karta zespołu	3
2	Analiza problemu	cel, interesariusze, zakres, persony	Opis problemu + zakres	4
3	Wymagania SRS	wymagania funkcjonalne i niefunkcjonalne	SRS v0.1 z REQ-xxx	6
4	UX i przypadki użycia	makiety, scenariusze, use case	Makiety + przypadki użycia	5
5	Architektura	baza danych, API, architektura, technologie	Diagramy + model danych	6
6	Sprint 1	pierwszy działający moduł	Demo części systemu	6
7	Sprint 1 Review	testy pierwszego modułu, poprawki	Demo + testy + aktualizacja dokumentacji	7
8	Sprint 2	MVP – podstawowe funkcjonalności	MVP systemu	7
9	Sprint 2 Review	stabilizacja MVP, błędy	Review MVP + backlog poprawek	6
10	Sprint 3	rozbudowa funkcji, role, raporty	Demo rozszerzeń	7
11	Testowanie	testy funkcjonalne i akceptacyjne	Tabela testów + wyniki	7
12	Bezpieczeństwo i jakość	walidacja, uprawnienia, błędy, UX	Lista poprawek + wersja RC	7
13	Release Candidate	uruchomienie wg instrukcji, finalizacja	RC + instrukcja instalacji	8
14	Audyt projektu	kompletność dokumentacji, Git, demo próbne	Kompletny pakiet projektu	8
15	Final Demo	prezentacja, obrona indywidualna	Prezentacja + system + dokumentacja	13

7. Punktacja projektu

Obszar	Maks. pkt	Komentarz
Analiza i wymagania	10	ocena na podstawie dowodów: dokumentacja, Git, demo, testy i obrona
Dokumentacja techniczna	15	ocena na podstawie dowodów: dokumentacja, Git, demo, testy i obrona
Architektura i projekt	10	ocena na podstawie dowodów: dokumentacja, Git, demo, testy i obrona
Implementacja	25	ocena na podstawie dowodów: dokumentacja, Git, demo, testy i obrona
Baza danych / API / integracja	10	ocena na podstawie dowodów: dokumentacja, Git, demo, testy i obrona
Testowanie i jakość	10	ocena na podstawie dowodów: dokumentacja, Git, demo, testy i obrona
Git, backlog i organizacja pracy	5	ocena na podstawie dowodów: dokumentacja, Git, demo, testy i obrona
System końcowy	5	ocena na podstawie dowodów: dokumentacja, Git, demo, testy i obrona
Prezentacja / demo	5	ocena na podstawie dowodów: dokumentacja, Git, demo, testy i obrona
Obrona indywidualna	5	ocena na podstawie dowodów: dokumentacja, Git, demo, testy i obrona

Mnożnik indywidualny: wynik zespołu może być pomnożony przez współczynnik wkładu studenta, np. 1,10 dla osoby bardzo aktywnej, 1,00 dla normalnego wkładu, 0,80 dla słabego wkładu, 0,50 dla osoby prawie nieaktywnej.

8. Bramki zaliczeniowe

- Gate 1 – Requirements Review

bez zaakceptowanych wymagań zespół nie przechodzi do właściwej implementacji

- Gate 2 – MVP Review

w połowie semestru musi działać podstawowa wersja produktu

- Gate 3 – Release Review

przed końcowym demo inna osoba musi móc uruchomić system według instrukcji

9. Prezentacja końcowa

- problem i cel systemu
- najważniejsze wymagania
- architektura i technologie
- pokaz działającej aplikacji
- testy i jakość
- problemy napotkane w trakcie pracy
- indywidualny wkład członków zespołu
- dalsze możliwości rozwoju

10. Minimalne standardy zaliczenia

- projekt uruchamia się według instrukcji
- istnieje repozytorium z historią pracy
- dokumentacja jest kompletna i zgodna z systemem
- każdy student potrafi wyjaśnić swój wkład

- system ma bazę danych, role użytkowników, podstawowe funkcje i testy
- brak pracy w ostatnim tygodniu nie zastępuje pracy semestralnej